

🎵 Manual de instalación

- 🎵 [Manual de instalación](#)
 - [Introducción](#)
 - [Herramientas](#)
 - [Pasos](#)
 - [Acceso](#)
 - [Pruebas](#)
 - [1. Prueba para una predicción alta:](#)
 - [2. Prueba para una predicción media:](#)
 - [3. Prueba para una predicción baja:](#)

Introducción

En este documento se detalla el proceso de despliegue de la aplicación, que consta de dos componentes principales: un dashboard y una API para un modelo de machine learning.

Ambas aplicaciones se encuentran contenerizadas para garantizar la portabilidad y consistencia entre los diferentes entornos. La API, que encapsula y sirve el modelo, está desarrollada con **FastAPI**, un moderno y rápido framework web para Python. El dashboard es una Aplicación de Página Única (SPA) construida con **Vue.js** y empaquetada con **Vite**, lo que proporciona una experiencia de usuario fluida y reactiva.

El desacoplamiento de estos componentes en servicios independientes ofrece varias ventajas clave:

1. **Escalabilidad Independiente:** Cada componente puede escalarse de forma horizontal o vertical según sus necesidades específicas de carga, sin afectar al otro.
2. **Flexibilidad Tecnológica:** Permite utilizar la pila tecnológica más adecuada para cada tarea (Python/FastAPI para la API, JavaScript/Vue para el frontend) y evolucionarlas de forma independiente.
3. **Desarrollo y Despliegue Autónomo:** Los equipos pueden trabajar en cada componente de manera paralela, y los despliegues se pueden realizar de forma independiente, lo que agiliza el ciclo de vida del desarrollo.

Herramientas

Para la construcción, el despliegue y la orquestación de la infraestructura se utilizan las siguientes herramientas:

- **Docker:** Es una plataforma de código abierto que permite a los desarrolladores crear, desplegar, ejecutar, actualizar y gestionar contenedores, que son componentes ejecutables estandarizados que

combinan el código fuente de la aplicación con las bibliotecas del sistema operativo (SO) y las dependencias necesarias para ejecutar ese código en cualquier entorno.

- **Python:** Es un lenguaje de programación de alto nivel, interpretado y de propósito general. Es conocido por su sintaxis clara y legible, lo que facilita la escritura de código. Se utiliza ampliamente en desarrollo web, ciencia de datos, inteligencia artificial y automatización.
- **JavaScript:** Es un lenguaje de programación que permite implementar funciones complejas en las páginas web. Se ejecuta en el lado del cliente (en el navegador) y se utiliza para crear contenido dinámico, controlar multimedia, animar imágenes y prácticamente todo lo que implique interacción del usuario.
- **Terraform:** Es una herramienta de infraestructura como código (IaC) de código abierto que permite a los usuarios definir y aprovisionar la infraestructura del centro de datos utilizando un lenguaje de configuración declarativo de alto nivel conocido como HashiCorp Configuration Language (HCL).
- **Kubernetes (usando EKS):** Kubernetes es un sistema de orquestación de contenedores de código abierto para automatizar la implementación, el escalado y la administración de aplicaciones en contenedores. Amazon Elastic Kubernetes Service (EKS) es un servicio administrado que facilita la ejecución de Kubernetes en AWS sin necesidad de instalar, operar y mantener su propio plano de control o nodos de Kubernetes.

Pasos

El despliegue se realiza siguiendo una serie de pasos automatizados y manuales. El archivo `Makefile` en la raíz del proyecto contiene acciones predefinidas que simplifican y estandarizan muchas de estas tareas, y puede tomarse como referencia para ejecutarlas.

1. Creación del Entorno con Terraform:

- Se utiliza Terraform para definir y aprovisionar la infraestructura necesaria en AWS, como el clúster de EKS, roles de IAM y repositorios ECR.
- Los comandos `init`, `plan` y `apply` del `Makefile` inicializan Terraform, crean un plan de ejecución y aplican los cambios para crear la infraestructura.

2. Creación de las Imágenes de Docker:

- Para cada aplicación (API y dashboard), se construye una imagen de Docker. Este proceso empaqueta el código de la aplicación y todas sus dependencias en un contenedor estandarizado.
- La acción `dkbuild` en el `Makefile` se encarga de este proceso, utilizando el `Dockerfile` correspondiente a cada aplicación.

3. Subida de Imágenes a Amazon ECR:

- Una vez construidas, las imágenes de Docker se suben a Amazon Elastic Container Registry (ECR), que es el registro de contenedores privado de AWS.
- La acción `dkpush` del Makefile automatiza el inicio de sesión en ECR y la subida de la imagen creada en el paso anterior.

4. **Despliegue en Amazon EKS:**

- Con la infraestructura lista y las imágenes en ECR, se procede a desplegar las aplicaciones en el clúster de Kubernetes (EKS).
- Se utilizan manifiestos de Kubernetes (archivos YAML) para definir los Deployments y Services de cada aplicación.
- La acción `eksapply` del Makefile configura `kubectl` para apuntar al clúster de EKS y aplica los manifiestos para desplegar los contenedores.

5. **Instalación de NGINX como Ingress Controller:**

- Para exponer las aplicaciones al tráfico externo de manera segura y controlada, se instala un Ingress Controller. En este caso, se utiliza NGINX.
- El Ingress enrutará las peticiones externas a los servicios correspondientes dentro del clúster (por ejemplo, `/api` a la API y `/` al dashboard).
- La acción `eksingress` del Makefile utiliza Helm para instalar el NGINX Ingress Controller en el clúster de EKS.

Acceso

El acceso a la aplicación se realiza a través de la URL expuesta usando el Ingress Controller. Debido a que el despliegue se ejecutó sobre una cuenta de AWS académica y con límite de acceso por tiempo, se debe solicitar la URL a los responsables del proyecto.

Pruebas

Una vez desplegada la API, puedes probar el endpoint de predicción utilizando `curl`. El siguiente comando envía una petición POST con un cuerpo JSON que sigue el schema `SongFeaturesInput`.

Reemplaza `[URL_DEL_API]` con la URL externa proporcionada por el Ingress Controller.

```
curl -X 'POST' \
  '[URL_DEL_API]/api/predict' \
  -H 'Content-Type: application/json' \
  -d '{
    "duration_ms": 250000,
    "explicit": false,
    "danceability": 0.75,
```

```

    "energy": 0.88,
    "key": 7,
    "loudness": -4.5,
    "mode": 1,
    "speechiness": 0.06,
    "acousticness": 0.12,
    "instrumentalness": 0.0001,
    "liveness": 0.18,
    "valence": 0.65,
    "tempo": 124.0,
    "time_signature": 4,
    "track_genre": "pop"
  }

```

1. Prueba para una predicción alta:

```

curl -X 'POST' \
  '[URL_DEL_API]/api/predict' \
  -H 'Content-Type: application/json' \
  -d '{
    "duration_ms": 210000,
    "explicit": false,
    "danceability": 0.35,
    "energy": 0.25,
    "key": 0,
    "loudness": -12.0,
    "mode": 1,
    "speechiness": 0.04,
    "acousticness": 0.85,
    "instrumentalness": 0.0000,
    "liveness": 0.12,
    "valence": 0.30,
    "tempo": 72.0,
    "time_signature": 3,
    "track_genre": "acoustic"
  }'

```

```

juan@fedora:~$ curl -X POST \
  "https://e89780a81548e1feddb97d0be2-1563421141.us-east-1.elb.amazonaws.com/api/predict" \
  -H 'Content-Type: application/json' \
  -d '{
    "duration_ms": 210000,
    "explicit": false,
    "danceability": 0.35,
    "energy": 0.25,
    "key": 0,
    "loudness": -12.0,
    "mode": 1,
    "speechiness": 0.04,
    "acousticness": 0.85,
    "instrumentalness": 0.0000,
    "liveness": 0.12,
    "valence": 0.30,
    "tempo": 72.0,
    "time_signature": 3,
    "track_genre": "acoustic"
  }'
{"class": "Alta"}

```

Predicción alta

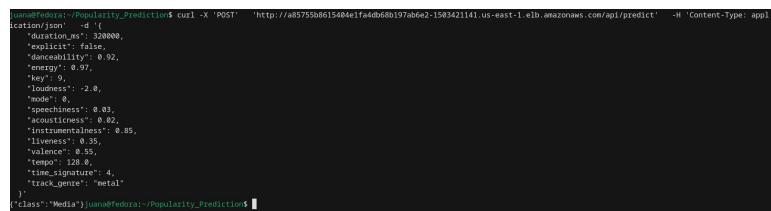
2. Prueba para una predicción media:

```

curl -X 'POST' \
  '[URL_DEL_API]/api/predict' \

```

```
-H 'Content-Type: application/json' \
-d '{
  "duration_ms": 320000,
  "explicit": false,
  "danceability": 0.92,
  "energy": 0.97,
  "key": 9,
  "loudness": -2.0,
  "mode": 0,
  "speechiness": 0.03,
  "acousticness": 0.02,
  "instrumentalness": 0.85,
  "liveness": 0.35,
  "valence": 0.55,
  "tempo": 128.0,
  "time_signature": 4,
  "track_genre": "metal"
}'
```



```
juan@fedora:~$ curl -X POST -H 'Content-Type: application/json' -d '{
  "duration_ms": 320000,
  "explicit": false,
  "danceability": 0.92,
  "energy": 0.97,
  "key": 9,
  "loudness": -2.0,
  "mode": 0,
  "speechiness": 0.03,
  "acousticness": 0.02,
  "instrumentalness": 0.85,
  "liveness": 0.35,
  "valence": 0.55,
  "tempo": 128.0,
  "time_signature": 4,
  "track_genre": "metal"
}' http://a0575b0615404e1fadbdb0197ab6e2-1583421141.us-east-1.elb.amazonaws.com/api/predict'
{"class": "Media", "juan@fedora:~$ curl -X POST -H 'Content-Type: application/json' -d '{
  "duration_ms": 320000,
  "explicit": false,
  "danceability": 0.92,
  "energy": 0.97,
  "key": 9,
  "loudness": -2.0,
  "mode": 0,
  "speechiness": 0.03,
  "acousticness": 0.02,
  "instrumentalness": 0.85,
  "liveness": 0.35,
  "valence": 0.55,
  "tempo": 128.0,
  "time_signature": 4,
  "track_genre": "metal"
}' http://a0575b0615404e1fadbdb0197ab6e2-1583421141.us-east-1.elb.amazonaws.com/api/predict'
{"class": "Media", "juan@fedora:~$
```

Predicción media

3. Prueba para una predicción baja:

```
curl -X 'POST' \
  '[URL_DEL_API]/api/predict' \
  -H 'Content-Type: application/json' \
  -d '{
    "duration_ms": 250000,
    "explicit": false,
    "danceability": 0.75,
    "energy": 0.88,
    "key": 7,
    "loudness": -4.5,
    "mode": 1,
    "speechiness": 0.06,
    "acousticness": 0.12,
    "instrumentalness": 0.0001,
    "liveness": 0.18,
    "valence": 0.65,
    "tempo": 124.0,
    "time_signature": 4,
    "track_genre": "pop"
  }'
```

```
juan@fedora:~/Popularity_Predictor$ curl -X 'POST' -H 'Content-Type: application/json' -d '{
  "duration_ms": 250000,
  "explicit": false,
  "danceability": 0.75,
  "energy": 0.88,
  "key": 7,
  "loudness": -4.5,
  "mode": 1,
  "speechiness": 0.06,
  "acousticness": 0.12,
  "instrumentalness": 0.0001,
  "liveness": 0.18,
  "valence": 0.55,
  "tempo": 124.0,
  "time_signature": 4,
  "track_genre": "pop"
}'
{"class": "Baja"}
juan@fedora:~/Popularity_Predictor$
```

Predicción baja